Projektova zprava
Nejlepsi tamagotchi na celym sigma svete

Matyas Potocký a Martin Veber

21. 4. 2026

Obsah

1 Zakladni informace **2**

2 Cil aplikace **2**

3 Pouzite technologie **2**

4 Architektura a struktura komponent **3**

5 Sprava stavu **5**

6 Nasazeni **6**

7 PWA **6**

8 Testovani **7**

9 Rozdeleni prace **9**

10 Zhodnoceni projektu **9**

11 Prilohy **10**

1 Zakladni informace

Nazev projektu: Nejlepsi tamagotchi na celym sigma svete

Jednoduche shrnuti: Jde o webovou hru, ve ktere si uzivatel vybere virtualniho mazlicka a potom se o nej stara. Mazlicek postupne ztraci energii, zdravi, hlad i stesti, hrac proto musi delat akce ve spravnou chvili a hlidat, aby zadny ukazatel neklesl na nulu.

Clenove tymu:

- Role A – UI: Martin Veber. Mel na starosti wireframe, mobilni rozlozeni, vizualni styl, animace a finalni vzhled obrazovek.
- Role B – logika: Matyas Potocký. Mel na starosti datove modely, stav aplikace, ukladani, nasazeni a podpurnou infrastrukturu.

Repozitar: https://github.com/cqeta1564/nejlepsi-tamagotchi-na-celym-sigma-svete

2 Cil aplikace

Cilem aplikace je vytvorit jednoduchou a srozumitelnou hru, ktera pripomina klasicke Tamagotchi. Uzivatel ma jednoho virtualniho mazlicka a musi se o nej starat tak, aby mu neklesly dulezite potreby na nulu. Aplikace je urcena hlavne pro studenty nebo bezne uzivatele, kteri si chteji rychle vyzkouset kratkou browserovou hru bez instalace a bez sloziteho ovladani.

Problem, ktery aplikace resi, neni nejaky vazny realny problem, ale spis herni situace: hrac chce mit prehlednou, rychlou a zabavnou peci o virtualniho mazlicka v jednom okne prohlizece. Aplikace proto uzivateli ukazuje jen to nejdulezitejsi: koho ma vybraneho, v jake mistnosti se prave nachazi, kolik ma penez a jak na tom mazlicek je. Vsechno je navrzene tak, aby tomu rozumel i nekdo, kdo neprogramuje.

3 Pouzite technologie

- React 19 a React DOM 19
- TypeScript 5.9
- Vite 8
- CSS v souboru `src/index.css`
- Tailwind CSS 4 jako soucast build pipeline
- PostCSS a Autoprefixer
- ESLint 9, `typescript-eslint`, `eslint-plugin-react-hooks`, `eslint-plugin-react-refresh`
- Node.js 24 v GitHub Actions workflow
- Git a GitHub
- GitHub Actions pro CI a deploy
- GitHub Pages pro hosting produkcni verze

- `localStorage` v prohlizeci pro obnovu rozehrane hry
- AI pomocniky pri vyvoji: pouziti GitHub Copilot/Codex asistovanych uprav

4 Architektura a struktura komponent

Rozdeleni aplikace

Aplikace je rozdelena na jeden hlavni kontejner `App`, dve obrazovky a vice mensich prezentacnich komponent. Vetsina dat a logiky vznika v `App.tsx`. Podrizene komponenty dostavaji data pres props a posilaji udalosti zpatky nahoru pres callback funkce.

Aktualni strom aktivnich komponent

- `App`
- `PetSelectionScreen`
- `PetSelectionList`
- `PetOptionCard`
- `PetAvatar`
- `CutoutImage`
- `SelectPetButton`
- `HomeScreen`
- `StatsPanel`
- `StatChip` (vnitrni pomocna komponenta uvnitr `StatsPanel`)
- `PetCard`
- `PetFace`
- `StatusModal`

Komponenty a jejich props

`App` Hlavni korenova komponenta. Rozhoduje, jestli se zobrazi vyber mazlicka, herni obrazovka nebo systemovy stav. Props nema. Drzi herni data, intervaly, uklada stav do `localStorage` a obsluhuje vsechny dulezite udalosti.

`PetSelectionScreen`
Obrazovka pro vyber mazlicka. Zobrazuje seznam mazlicku a tlacitko pro potvrzeni vyberu. Prijima props `pets`, `selectedPetId`, `onSelectPet`, `onConfirmSelection`.

`PetSelectionList`
Kontejner, ktery vyrenderuje karty vsech dostupnych mazlicku. Prijima props `pets`, `selectedPetId`, `onSelect`.

`PetOptionCard`
: Jedna karta ve vyberu mazlicka. Zobrazuje obrazek a jmeno. Prijima props `pet`, `isSelected`, `onSelect`. Po kliknuti vola callback s id mazlicka.

`SelectPetButton`
: Tlacitko pro potvrzeni vyberu. Prijima props `disabled` a `onClick`.

`HomeScreen`
: Hlavni herni obrazovka po startu hry. Zobrazuje horni listu, staty a hlavni kartu s mistnosti. Prijima props `pet`, `roomId`, `roomName`, `roomDescription`, `roomBackgroundImage`, `actionIcon`, `actionLabel`, `actionCost`, `coins`, `statusText`, `onPrevRoom`, `onNextRoom`, `onRoomAction`, `onRestartRequest`, `isActionDisabled`.

`StatsPanel`
: Panel se ctyrmi ukazateli stavu mazlicka. Prijima props `stats`. Vnitrne si pro kazdy ukazatel vytvori komponentu `StatChip`.

`StatChip`
: Pomocna komponenta uvnitr `StatsPanel`. Zobrazuje nazev statistiky, ciselnou hodnotu a vizualni zaplneni. Prijima props `label`, `value`, `tone`.

`PetCard`
: Nejvetsi prezentacni karta herni obrazovky. Zobrazuje aktualni mistnost, penize, samotneho mazlicka, statusovou zpravu a ovladaci tlacitka. Prijima props `pet`, `roomId`, `roomName`, `roomDescription`, `roomBackgroundImage`, `actionIcon`, `actionLabel`, `actionCost`, `coins`, `statusText`, `onPrevRoom`, `onNextRoom`, `onRoomAction`, `isActionDisabled`.

`PetAvatar`
: Obaluje obrazek mazlicka a resi fallback, kdyz se obrazek nepovede nacist. Prijima props `src`, `alt`, `children`. V herni obrazovce do sebe vklada komponentu `PetFace`.

`PetFace`
: Kresli oblicej mazlicka pres SVG podle aktualnich statistik. Prijima props `petId` a `stats`.

`CutoutImage`
: Pomocna komponenta pro nacteni obrazku a odstraneni svetleho okraje pomoci canvasu. Prijima props `src`, `alt`, `className`, volitelne `onError`.

`StatusModal`
: Modal pro restart nebo game over. Prijima props `isOpen`, `title`, `message`, `variant`, `onClose`, `onConfirm`.

Tok dat a spoluprace komponent

Zdroj puvodnich dat je v `src/data/pets.ts`, kde jsou definovani tri ukazkovi mazlicci. `App` z techto dat vytvori pocatecni stav hry a dale drzi:

- `gameState` – aktualni obrazovku, vybraneho mazlicka a seznam vsech mazlicku,
- `roomIndex` – aktualne vybranou mistnost,
- `coins` – pocet penez,
- `statusText` – posledni informacni zpravu pro hrace,

- `modalState` – stav modalniho okna.

Data potom putuji smerem dolu takto:

- `App -> PetSelectionScreen -> PetSelectionList -> PetOptionCard`
- `App -> HomeScreen -> StatsPanel`
- `App -> HomeScreen -> PetCard -> PetAvatar -> PetFace`
- `App -> StatusModal`

Udalosti se vraceji zpet nahoru pres callbacky:

- klik na `PetOptionCard` zavola `onSelect`, to se prenese az do `App.handleSelectPet()`,
- klik na `SelectPetButton` zavola `App.handleConfirmSelection()`,
- klik na sipky v `PetCard` zavola `App.handlePrevRoom()` nebo `App.handleNextRoom()`,
- klik na hlavni akcni tlacitko v `PetCard` zavola `App.handleRoomAction()`,
- klik na restart zavola `App.handleRequestRestart()` a nasledne pres modal `App.handleRestartGa`

5 Sprava stavu

Hlavni stav je spravovan pomoci `useState`, `useMemo`, `useEffect` a `useRef`. Aplikace si uklada rozehranou hru do `localStorage` pod klicem `tamagotchi-wireframe-state`.

Ukladany format je JSON objekt ve tvaru:

```
{
  "gameState": {
    "currentScreen": "home",
    "selectedPetId": "pet-1",
    "pets": [ ... ]
  },
  "roomIndex": 0,
  "coins": 24,
  "statusText": "Kuchyn je pripravena..."
}
```

Pri nacitani aplikace probehne tento postup:

1. Po mountu `App` zkontroluje, jestli je v `localStorage` ulozeny predchozi stav.
2. Pokud data existuji a maji spravny tvar, aplikace je nacte do stavu.
3. Pokud jsou data rozbita nebo nevalidni, zobrazi se systemova chyba a uzivatel muze hru restartovat.

Pri bezne akci ve hre je tok stavu nasledujici:

1. Uzivatel klikne na akcni tlacitko v aktualni mistnosti.
2. Zavola se `handleRoomAction()`.
3. Funkce zkontroluje, jestli je mazlicek nazivu a jestli ma hrac dostatek penez.

4. Nasledne se spocita novy stav statistik pomoci `executeRoomAction()`.
5. `setGameState()` prepise konkretniho mazlicka v poli `pets`.
6. `setCoins()` upravi pocet penez a `setStatusText()` prepise informacni zpravu.
7. React komponentu prekresli a uzivatel okamzite vidi novy stav.
8. Vedle toho se zmeneny stav automaticky znovu ulozi do `localStorage`.

Podobne funguje i casovana herni smycka:

- kazdych 12 sekund se spusti interval, ktery snizi `food`, `health`, `happiness` a `energy`,
- kazdych 5 sekund se pricte 1 mince,
- kdyz libovolny stat klesne na 0, otevira se modal „Mazlicek zemrel“ a hra se musi restartovat.

6 Nasazeni

Aplikace bezi na GitHub Pages:

https://cqeta1564.github.io/nejlepsi-tamagotchi-na-celym-sigma-svete/

Standardni build prikaz v projektu je:

```
npm run build
```

Pri uspesnem buildu vznika vystup ve slozce `dist/`. Tato slozka obsahuje zejmena:

- `dist/index.html`
- `dist/assets/index-*.js`
- `dist/assets/index-*.css`
- staticke assety, napriklad `dist/favicon.svg`

Nasazeni je automatizovane pres workflow `.github/workflows/deploy-pages.yml`:

1. po pushi do vetve `main` se spusti GitHub Actions,
2. workflow nainstaluje zavislosti prikazem `npm ci`,
3. spusti build,
4. adresar `dist/` nahraje pomoci `actions/upload-pages-artifact`,
5. nasledne ho pomoci `actions/deploy-pages` publikuje na GitHub Pages.

Vite je nastaveno s pevnou `base` cestou `/nejlepsi-tamagotchi-na-celym-sigma-svete/`, aby spravne fungovaly cesty pri nasazeni pod repozitarem.

7 PWA

Aktualni verze projektu neni plnohodnotna PWA.

V repozitari se nenachazi:

- registrace service workeru v `src/main.tsx`,
- soubor typu `manifest.webmanifest`,
- vlastni cache strategie pro offline provoz.

To v praxi znamena:

- pri instalaci se nesestavuje specialni PWA cache,
- sitove pozadavky se neresí pres service worker, ale standardne pres prohlizec,
- pokud uzivatel nema pripojeni a stranku znovu otevira nebo obnovi, aplikace nebude spolehlive dostupna offline.

Z pohledu Lighthouse by proto PWA audit v aktualnim stavu neprosel. Pravdepodobne by neprosly hlavne podminky spojene s web app manifestem, service workerem a offline podporou. Pokud uzivatel nechal aplikaci otevrenou jeste pred ztratou site, muze v ni kratce pokracovat, ale po obnoveni stranky uz se bez pripojeni nespusti.

Obrázek 1: Placeholder pro Lighthouse audit

8 Testovani

Automaticke testy v repozitari momentalne nejsou. Overovani probiha hlavne rucne a doplnkove pres CI, ktere spousti lint a build. Pri lokalni kontrole 21. 4. 2026 dopadly tyto kontroly takto:

- `npm run lint` – neproslo, ESLint hlasi problem v `src/components/CutoutImage.tsx` kvuli synchronnimu `setState` uvnitr `useEffect`,
- `npm run build` – neproslo, protoze chybi modul `@tailwindcss/postcss` v lokalnim prostredi.

Rucne prosle testovaci scenare:

Scenar 1 – prazdny start bez ulozene hry

- Vstupni podminka: v `localStorage` neni zadny klic `tamagotchi-wireframe-state`.
- Kroky: otevru aplikaci poprve.
- Ocekavany vysledek: zobrazi se obrazovka pro vyber mazlicka a tlacitko pro potvrzeni bude na zacatku neaktivni.
- Skutecny vysledek: odpovida ocekavani.

Scenar 2 – pokus pokracovat bez vyberu mazlicka

- Vstupni podminka: aplikace je na obrazovce vyberu, zadny mazlicek neni oznacen.
- Kroky: pokusim se pokracovat bez predchoziho vyberu.
- Ocekavany vysledek: tlacitko „potvrdit vyber“ zustane zablokovane a hra se nespusti.

- Skutecny vysledek: odpovida ocekavani.

Scenar 3 – vyber mazlicka a start hry

- Vstupni podminka: aplikace je na uvodni obrazovce.
- Kroky: vyberu jednoho mazlicka a kliknu na „potvrdit vyber“.
- Ocekavany vysledek: zobrazi se herni obrazovka, aktualni mistnost je kuchyn a hrac ma 24 penez.
- Skutecny vysledek: odpovida ocekavani.

Scenar 4 – provedeni akce v mistnosti

- Vstupni podminka: hra bezi, mazlicek je nazivu a hrac ma dostatek penez.
- Kroky: v kuchyni kliknu na tlacitko akce.
- Ocekavany vysledek: odectou se penize, zmeni se statistiky mazlicka a prepise se statusova zprava.
- Skutecny vysledek: odpovida ocekavani.

Scenar 5 – nedostatek penez

- Vstupni podminka: hra bezi, ale hrac nema dost penez na aktualni akci.
- Kroky: prepnu se do drazsi mistnosti a pokusim se akci spustit.
- Ocekavany vysledek: akcni tlacitko bude neaktivni, aby neslo utratit vic penez, nez je k dispozici.
- Skutecny vysledek: odpovida ocekavani.

Scenar 6 – casovy pokles statu

- Vstupni podminka: hra bezi na domovske obrazovce.
- Kroky: nic neudelam a pockam alespon 12 sekund.
- Ocekavany vysledek: hlad, zdravi, stesti a energie klesnou podle hernich pravidel a zmeni se statusova zprava.
- Skutecny vysledek: odpovida ocekavani.

Scenar 7 – game over

- Vstupni podminka: nektery stat mazlicka je tesne nad nulou.
- Kroky: necham bezet hru, dokud jeden stat neklesne na 0.
- Ocekavany vysledek: otevre se modal s informaci o smrti mazlicka a hra vyzaduje restart.
- Skutecny vysledek: odpovida ocekavani.

Scenar 8 – obnova po reloadu

- Vstupni podminka: hra je rozehrana a v `localStorage` je ulozen stav.
- Kroky: obnovim stranku v prohlizeci.
- Ocekavany vysledek: aplikace nacte predchoziho mazlicka, mistnost, penize i posledni stav hry.
- Skutecny vysledek: odpovida ocekavani.

Scenar 9 – rozbita ulozena data

- Vstupni podminka: v `localStorage` je schvalne vlozen nevalidni JSON nebo spatny tvar dat.
- Kroky: obnovim stranku.
- Ocekavany vysledek: zobrazi se systemova chyba a tlacitko pro novy zacatek.
- Skutecny vysledek: odpovida ocekavani.

Scenar 10 – chyba site / offline stav

- Vstupni podminka: uzivatel odpoji internet a chce aplikaci otevrit znovu.
- Kroky: vypnu pripojeni a zkusim obnovit nebo znovu otevrit nasazenou aplikaci.
- Ocekavany vysledek: protoze nejde o plnou PWA, aplikace nebude po refreshi spolehlive offline fungovat.
- Skutecny vysledek: odpovida ocekavani, offline provoz neni implementovan.

9 Rozdeleni prace

Z historie vetvi a commit je videt, ze prace byla delena paralelne mezi wireframe/UI cast a logickou/infrastrukturni cast:

- Martin Veber (Role A – UI): priprava wireframu, barevneho stylu, mobilniho layoutu, pet scen, HUD prvku, animaci a finalniho vizualniho poladeni.
- Matyas Potocký (Role B – logika): priprava datovych typu, vyberu mazlicka, spravy stavu, ukladani do `localStorage`, casove herni smycky, CI, branch protection a nasazeni na GitHub Pages.
- Obe role na sebe navazovaly: nejdriv vznikl wireframe a komponentovy zaklad, potom se doplnila herni logika, persistence a deploy, a nakonec se vzhled znovu doladil.

10 Zhodnoceni projektu

Co se podarilo

- Podarilo se vytvorit funkcni browserovou hru, ktera ma jasny herni cil a jednoduche ovladani.

- Vysledna aplikace pusobi vizualne jednotne a mobile-first rozlozeni je na tak maly projekt nadprumerne propracovane.
- Dobre funguje spojeni logiky a UI: zmeny statu se okamzite projevuji v obraze mazlicka, penizich i informacni hlasce.
- Prakticka je i persistence do `localStorage`, diky ktere uzivatel o rozehranou hru neprijde po refreshi.
- Pozitivni je take to, ze projekt ma CI workflow a verejne nasazeni na GitHub Pages.

Co bylo problematicke

- V repozitari zustaly starsi nepouzivane komponenty z drivejsi iterace, takze kodova zakladna neni uplne uklizena.
- PWA cast zadani zatim neni splnena. Chybi service worker, manifest i offline rezim.
- V aktualnim lokalnim prostredi neprochazi lint ani build, coz ukazuje, ze by bylo dobre dotahnout zavislosti a odstranit chybu v `CutoutImage`.
- Projekt zatim nema automaticke testy, takze spoleha hlavne na rucni kontrolu a zakladni CI kroky.

Co by slo dale rozvijet

- Dopsat skutecnou PWA vrstvu: manifest, service worker, offline fallback a instalaci na plochu.
- Pridat automaticke testy, idealne jednotkove testy herni logiky a zakladni integracni testy hlavnich obrazovek.
- Rozsirit hru o dalsi mistnosti, vice typu akci, inventar nebo vice druhu mazlicku.
- Pridat zvuky, denni odmeny, skore nebo statistikou delky preziti.
- Uklidit starsi nepouzivane komponenty a sjednotit dokumentaci s aktualni implementaci.

11 Prilohy

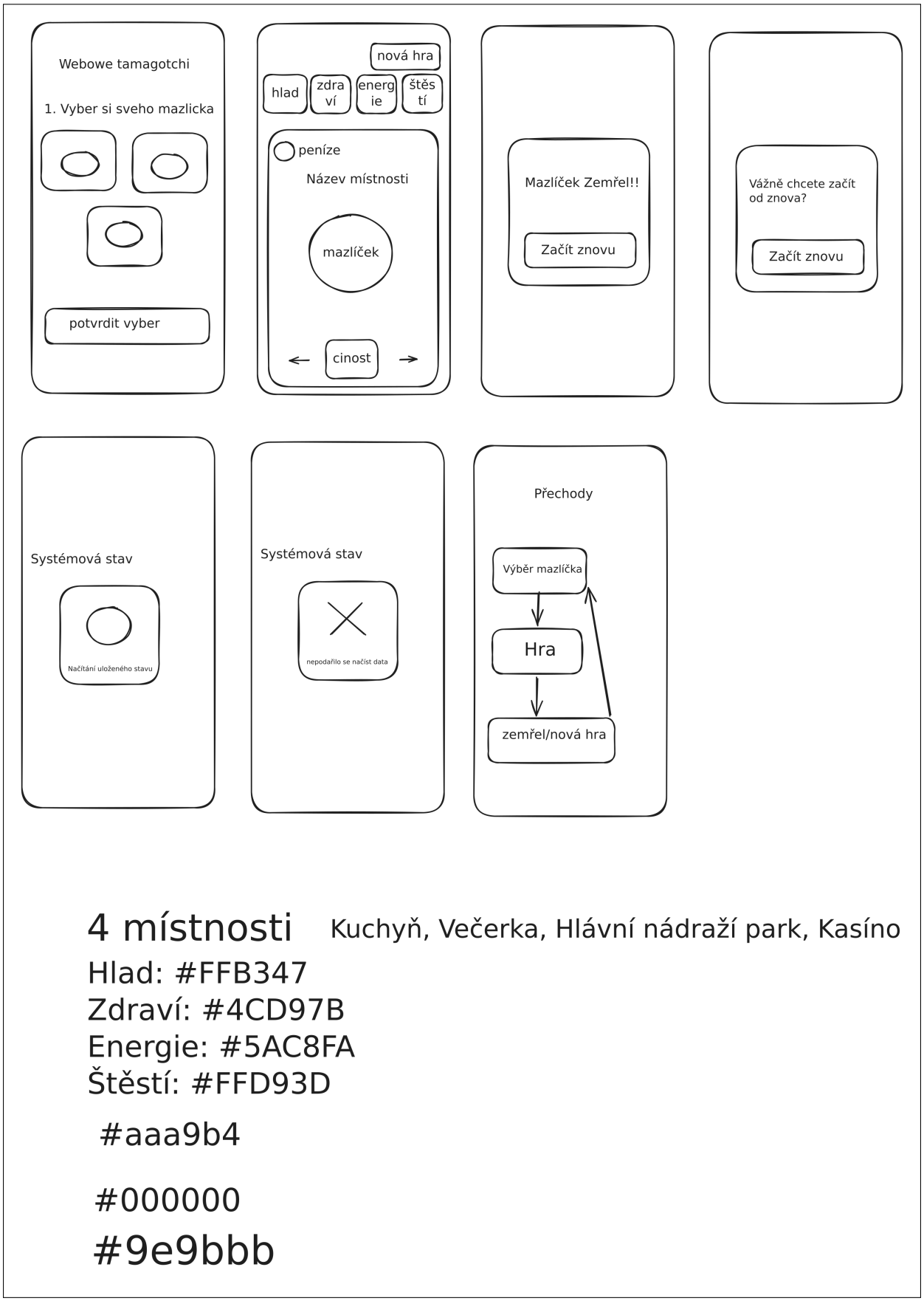
Wireframe z prvniho tydne

Puvodni wireframe je ulozen v repozitari jako `docs/wireframe.svg`.

Screenshoty finalni aplikace

V repozitari zatim nejsou ulozene screenshoty finalni mobilni ani desktopove verze. Pro finalni odevzdani doporucuji doplnit alespon dva obrazky:

- mobilni zobrazeni vyberu mazlicka nebo herni obrazovky,
- desktopove zobrazeni stejne obrazovky pro srovnani.



Obrázek 2: Wireframe z prvniho tydne

Doplnte screenshot z mobilni sirky, idealne herni obrazovku s viditelnymi staty a mistnosti.

Obrázek 3: Finalni aplikace – mobil

Doplnte screenshot z desktopove sirky, aby bylo videt centrovane rozlozeni aplikace.

Obrázek 4: Finalni aplikace – desktop